1. Create an S3 Bucket (for later operations)

What it does:

Creates an S3 bucket in AWS. This will be our base bucket for all other operations.

Why it's used:

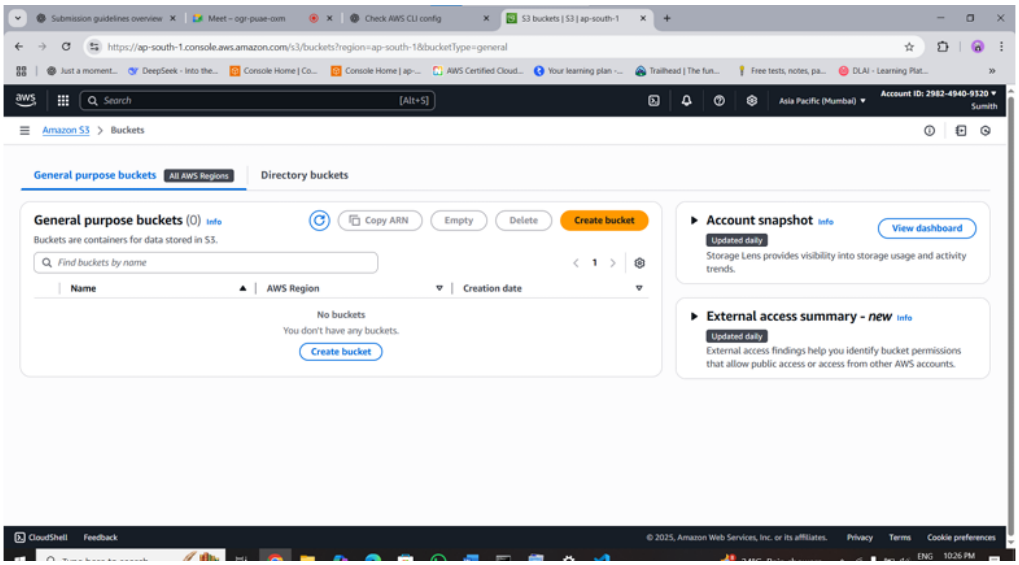
Buckets are the root container for storing files (objects) in AWS S3. Without a bucket, we can't do anything.

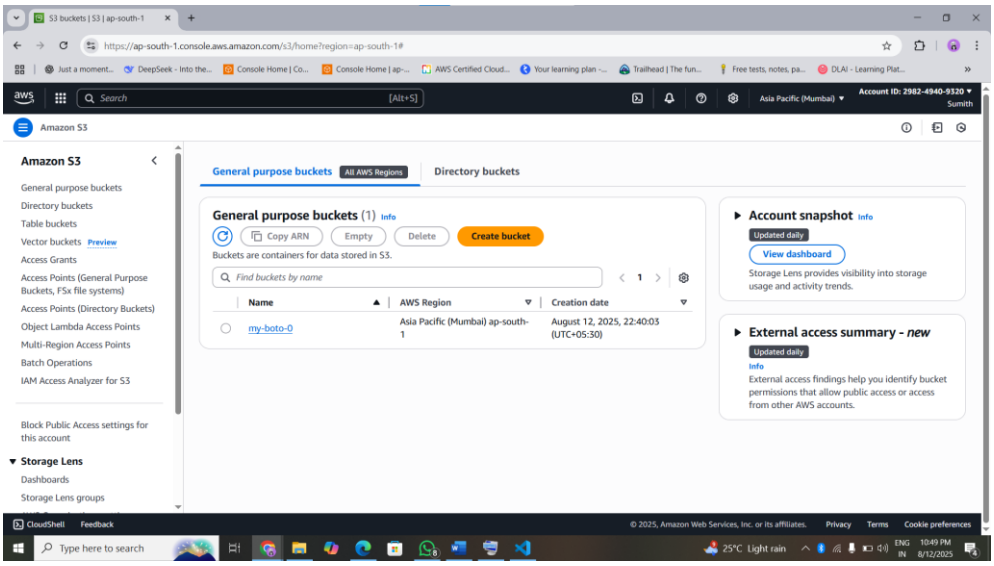
Steps:

1. Import boto3 and configure AWS credentials.
2. Use create_bucket method.
3. Specify region and unique bucket name.

Code:

```
import boto3
# Configuration
REGION = "ap-south-1"  # Change region if needed
BUCKET_NAME = "my-boto-0"  # Must be unique
# Create S3 client
s3_client = boto3.client("s3", region_name=REGION)
# Create bucket
s3_client.create_bucket(
    Bucket=BUCKET_NAME,
    CreateBucketConfiguration={"LocationConstraint": REGION}
)
print(f"Bucket '{BUCKET_NAME}' created successfully in {REGION}.")
```





---

2. Static Website Hosting (put_bucket_website)

What it does:

Turns an S3 bucket into a static website host.

Why it's used:

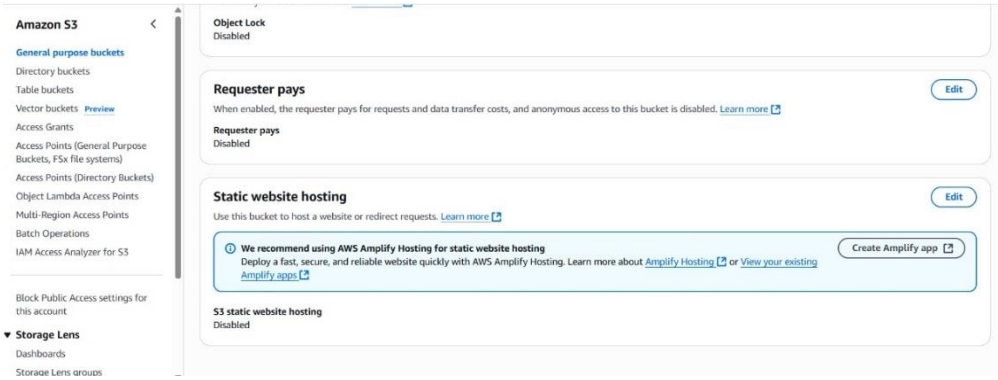
Host static HTML/CSS/JS files directly from S3 without a web server.

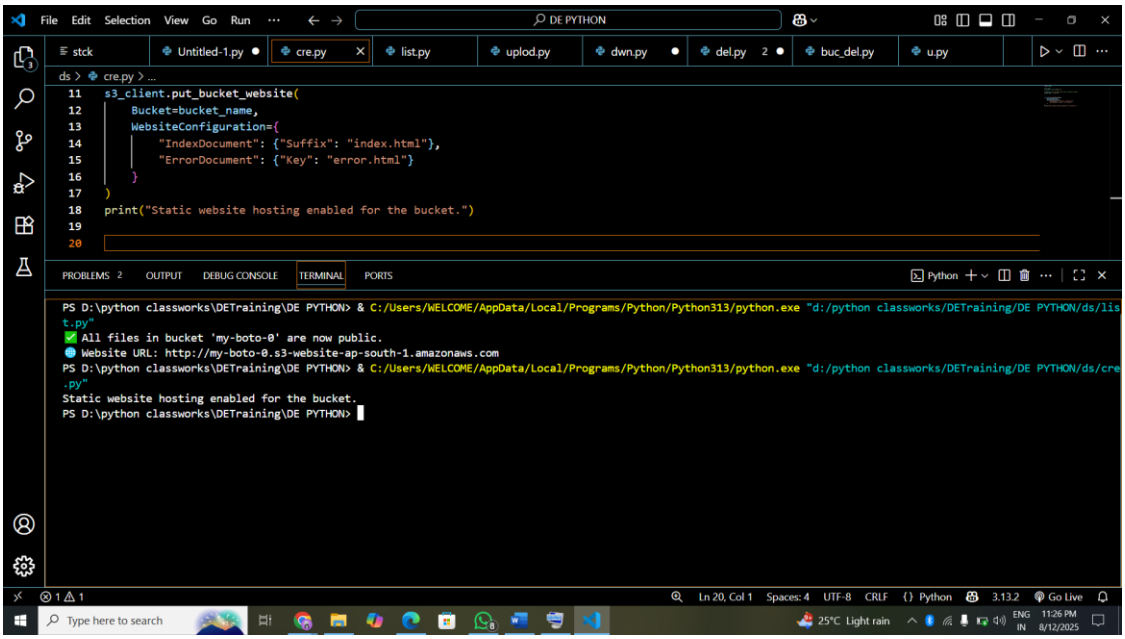
Steps:

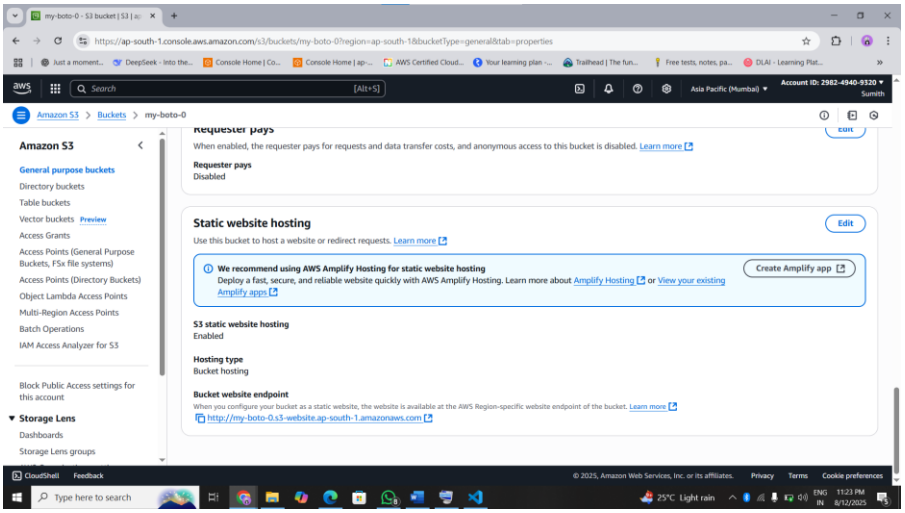
1. Have an existing bucket.
2. Upload index.html and error.html (optional).
3. Use put_bucket_website to set index and error pages.

Code:

```
import boto3
# S3 client
s3_client = boto3.client('s3')
# Replace with your bucket name (must be globally unique)
bucket_name = "my-boto-0"
s3_client.put_bucket_website(
    Bucket=bucket_name,
    WebsiteConfiguration={
        "IndexDocument": {"Suffix": "index.html"},
        "ErrorDocument": {"Key": "error.html"}
    }
)
print("Static website hosting enabled for the bucket.")
```


Amazon S3
General purpose buckets
Directory buckets
Table buckets
Vector buckets Preview
Access Grants
Access Points (General Purpose Buckets, FSx file systems)
Access Points (Directory Buckets)
Object Lambda Access Points
Multi-Region Access Points
Batch Operations
IAM Access Analyzer for S3
Block Public Access settings for this account
Storage Lens
Dashboards
Storage Lens groups
Object Lock
Disabled
Requester pays
When enabled, the requester pays for requests and data transfer costs, and anonymous access to this bucket is disabled. Learn more
Edit
Requester pays
Disabled
Static website hosting
Use this bucket to host a website or redirect requests. Learn more
Edit
We recommend using AWS Amplify Hosting for static website hosting
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. Learn more about Amplify Hosting or View your existing Amplify apps
Create Amplify app
S3 static website hosting
Disabled


File Edit Selection View Go Run
DE PYTHON
stck
Untitled-1.py
cre.py
list.py
uplod.py
dwn.py
del.py 2
buc_del.py
u.py
ds > cre.py > ...
11 s3_client.put_bucket_website(
12 Bucket=bucket_name,
13 WebsiteConfiguration={
14 "IndexDocument": {"Suffix": "index.html"},
15 "ErrorDocument": {"Key": "error.html"}
16 }
17)
18 print("Static website hosting enabled for the bucket.")
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/list.py"
All files in bucket 'my-boto-0' are now public.
Website URL: http://my-boto-0.s3-website-ap-south-1.amazonaws.com
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
Static website hosting enabled for the bucket.
PS D:\python classworks\DETraining\DE PYTHON>
Ln 20, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.13.2 Go Live
Type here to search
25°C Light rain
ENG IN 11:26 PM 8/12/2025


Amazon S3 > Buckets > my-boto-0
Amazon S3
General purpose buckets
Directory buckets
Table buckets
Vector buckets Preview
Access Grants
Access Points (General Purpose Buckets, FSx file systems)
Access Points (Directory Buckets)
Object Lambda Access Points
Multi-Region Access Points
Batch Operations
IAM Access Analyzer for S3
Block Public Access settings for this account
Storage Lens
Dashboards
Storage Lens groups
Requester pays
When enabled, the requester pays for requests and data transfer costs, and anonymous access to this bucket is disabled. Learn more
Requester pays
Disabled
Static website hosting
Use this bucket to host a website or redirect requests. Learn more
Edit
We recommend using AWS Amplify Hosting for static website hosting
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. Learn more about Amplify Hosting or View your existing Amplify apps
Create Amplify app
S3 static website hosting
Enabled
Hosting type
Bucket hosting
Bucket website endpoint
When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. Learn more
http://my-boto-0.s3-website.ap-south-1.amazonaws.com

3. Configure Public Access (put_bucket_acl & put_public_access_block)

What it does:

Sets bucket ACL and removes public access blocks.

Why it's used:

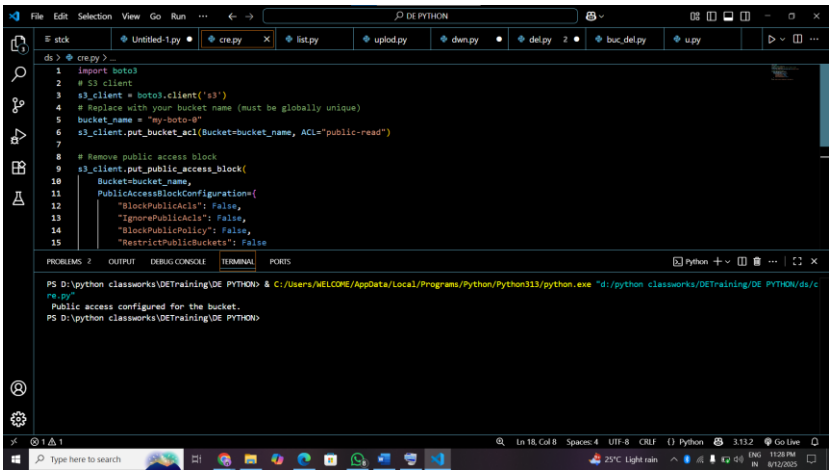
Required if you want your website or files to be publicly viewable.

Steps:

1. Allow bucket ACL to be public-read.
2. Remove public access restrictions.

Code:

```
import boto3
# S3 client
s3_client = boto3.client('s3')
# Replace with your bucket name (must be globally unique)
bucket_name = "my-boto-0"
s3_client.put_bucket_acl(Bucket=bucket_name, ACL="public-read")
# Remove public access block
s3_client.put_public_access_block(
    Bucket=bucket_name,
    PublicAccessBlockConfiguration={
        "BlockPublicAcls": False,
        "IgnorePublicAcls": False,
        "BlockPublicPolicy": False,
        "RestrictPublicBuckets": False
    }
)
print(" Public access configured for the bucket.")
```



---

4. Enable Versioning (put_bucket_versioning)

What it does:

Keeps multiple versions of objects.

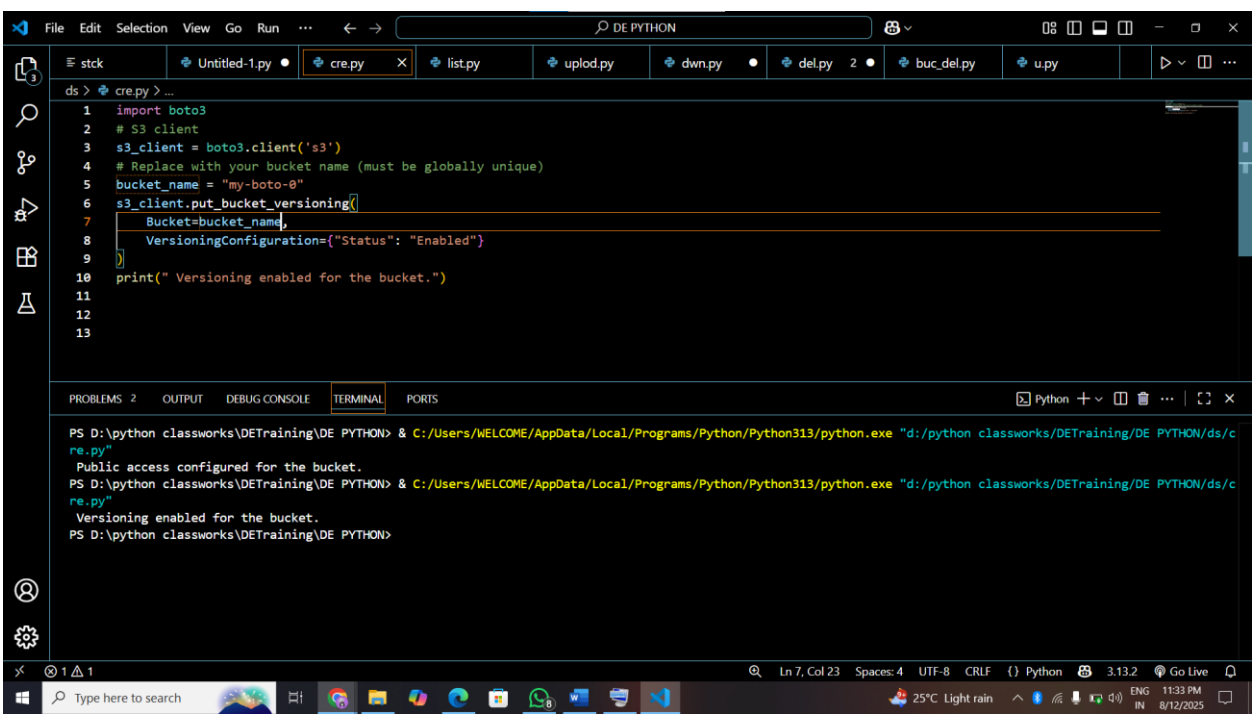
Why it's used:

Allows you to restore older versions of files.

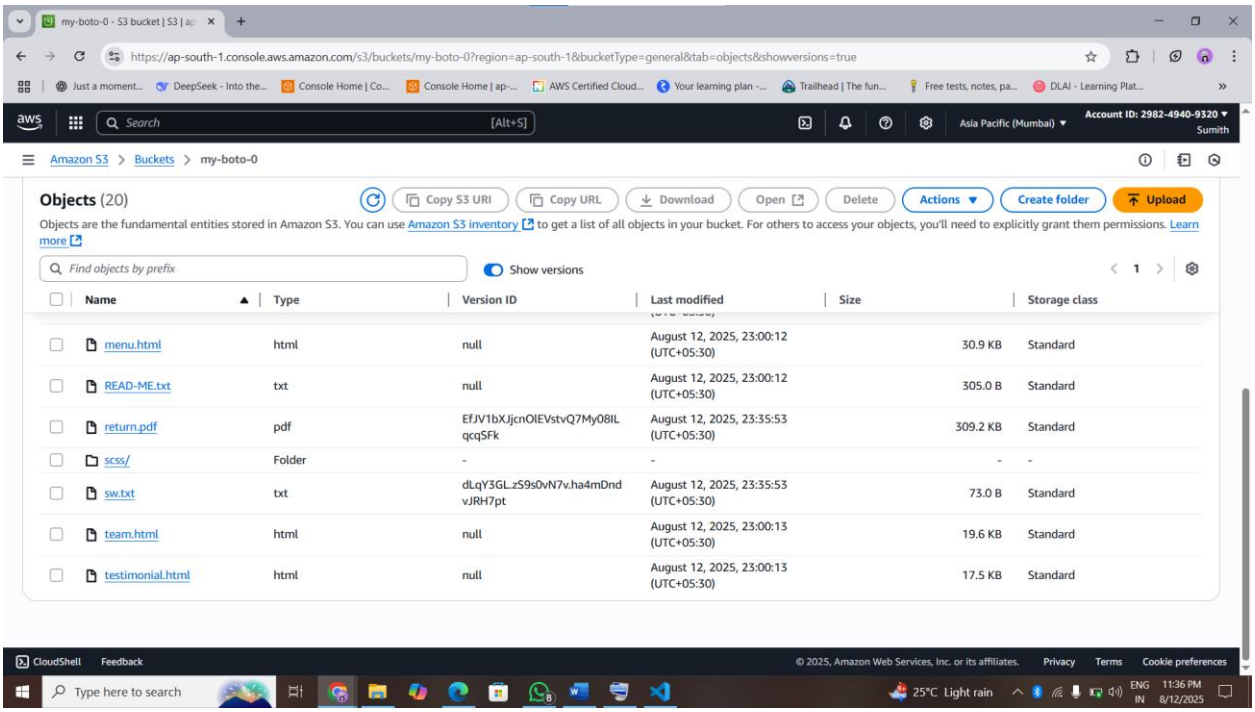
Steps:

1. Enable versioning with Status=Enabled.
2. Check in AWS Console.

Code:



```
import boto3
# S3 client
s3_client = boto3.client('s3')
# Replace with your bucket name (must be globally unique)
bucket_name = "my-boto-0"
s3_client.put_bucket_versioning(
    Bucket=bucket_name,
    VersioningConfiguration={"Status": "Enabled"}
)
print(" Versioning enabled for the bucket.")
```



---

5. Set Lifecycle Policies (put_bucket_lifecycle_configuration)

What it does:

Automatically deletes or transitions objects after a set time.

Why it's used:

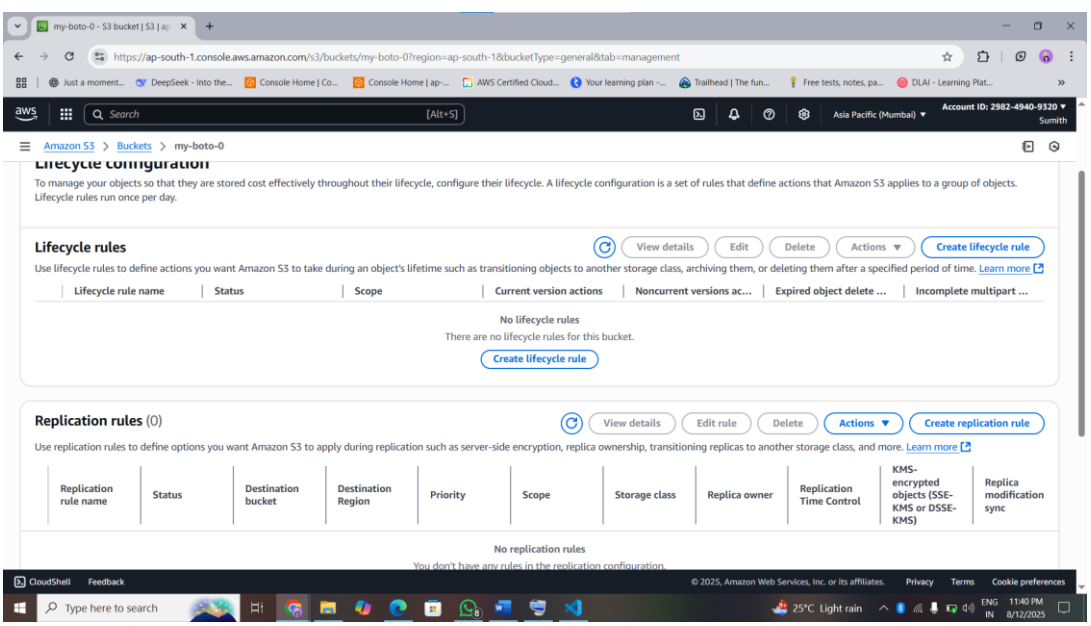
Reduces storage costs.

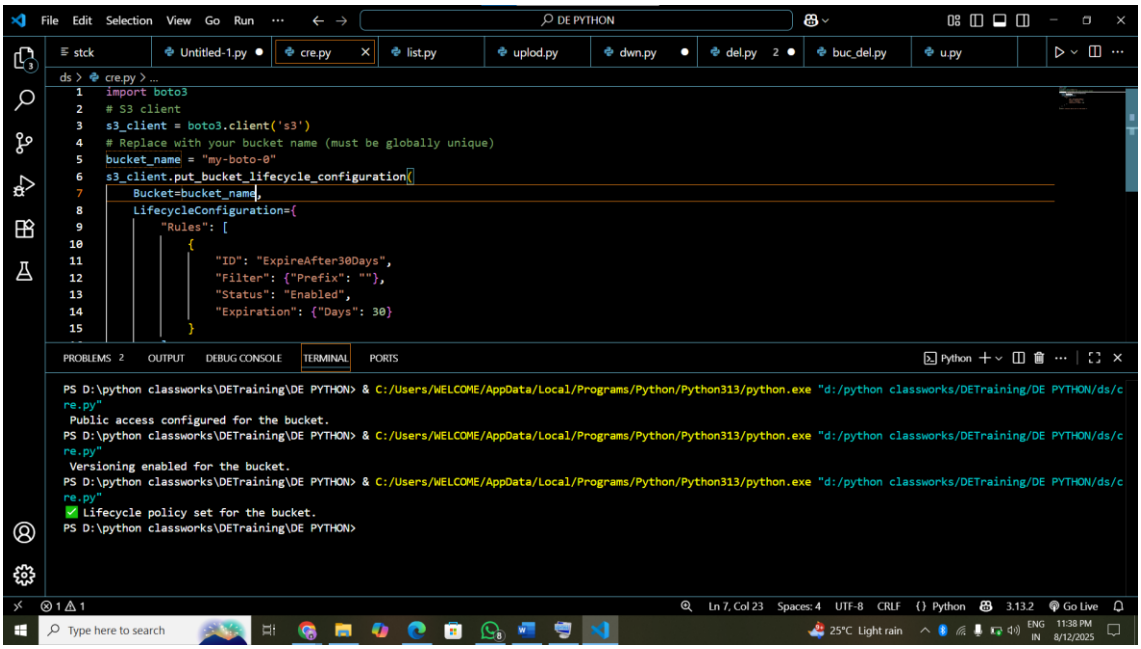
Steps:

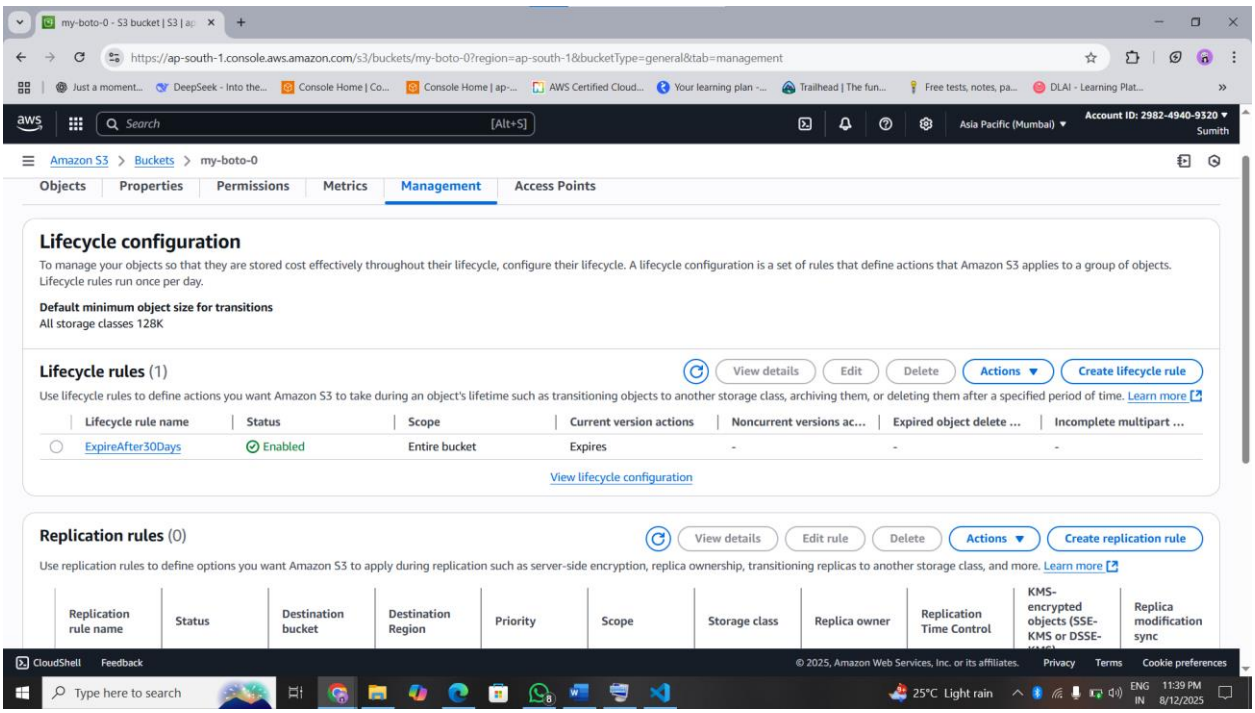
1. Create a lifecycle rule.
2. Apply using put_bucket_lifecycle_configuration.

Code:

```
import boto3
# S3 client
s3_client = boto3.client('s3')
# Replace with your bucket name (must be globally unique)
bucket_name = "my-boto-0"
s3_client.put_bucket_lifecycle_configuration(
    Bucket=bucket_name,
    LifecycleConfiguration={
        "Rules": [
            {
                "ID": "ExpireAfter30Days",
                "Filter": {"Prefix": ""},
                "Status": "Enabled",
                "Expiration": {"Days": 30}
            }
        ]
    }
)
print(" Lifecycle policy set for the bucket.")
```




File Edit Selection View Go Run
DE PYTHON
stck
Untitled-1.py
cre.py
list.py
uplod.py
dwn.py
del.py
buc_del.py
u.py
ds > cre.py > ...
import boto3
# S3 client
s3_client = boto3.client('s3')
# Replace with your bucket name (must be globally unique)
bucket_name = "my-boto-0"
s3_client.put_bucket_lifecycle_configuration(
Bucket=bucket_name,
LifecycleConfiguration={
"Rules": [
{
"ID": "ExpireAfter30Days",
"Filter": {"Prefix": ""},
"Status": "Enabled",
"Expiration": {"Days": 30}
}
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
Public access configured for the bucket.
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
Versioning enabled for the bucket.
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
Lifecycle policy set for the bucket.
PS D:\python classworks\DETraining\DE PYTHON>
Ln 7, Col 23
Spaces: 4
UTF-8
CRLF
Python
3.13.2
Go Live
Type here to search
25°C Light rain
ENG IN
11:38 PM
8/12/2025


my-boto-0 - S3 bucket | S3 | ap
https://ap-south-1.console.aws.amazon.com/s3/buckets/my-boto-0?region=ap-south-1&bucketType=general&tab=management
Search
[Alt+S]
Asia Pacific (Mumbai)
Account ID: 2982-4940-9320
Sumith
Amazon S3 > Buckets > my-boto-0
Objects
Properties
Permissions
Metrics
Management
Access Points
Lifecycle configuration
To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.
Default minimum object size for transitions
All storage classes 128K
Lifecycle rules (1)
View details
Edit
Delete
Actions
Create lifecycle rule
Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. Learn more
Lifecycle rule name
Status
Scope
Current version actions
Noncurrent versions ac...
Expired object delete ...
Incomplete multipart ...
ExpireAfter30Days
Enabled
Entire bucket
Expires
View lifecycle configuration
Replication rules (0)
View details
Edit rule
Delete
Actions
Create replication rule
Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. Learn more
Replication rule name
Status
Destination bucket
Destination Region
Priority
Scope
Storage class
Replica owner
Replication Time Control
KMS-encrypted objects (SSE-KMS or DSSE-
Replica modification sync
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Type here to search
25°C Light rain
ENG IN
11:39 PM
8/12/2025

6. Cross-Region Replication (put_bucket_replication)

What it does:

Replicates data from one bucket to another in a different AWS region.

Why it's used:

For disaster recovery and low-latency access.

Steps:

1. Create an IAM role with replication permissions.
2. Create a target bucket in another region.
3. Apply replication config.

Code:

```
s3_client.put_bucket_replication(
    Bucket=my-boto-0,
    ReplicationConfiguration={
        "Role": "arn:aws:iam::123456789012:role/ReplicationRole",
        "Rules": [
            {
                "ID": "ReplicationRule1",
                "Status": "Enabled",
                "Filter": {"Prefix": ""},
                "Destination": {
                    "Bucket": "arn:aws:s3:::my-target-boto",
                    "StorageClass": "STANDARD"
                }
            }
        ]
    }
)
print(" Cross-region replication configured.")
```

File Edit Selection View Go Run ... DE PYTHON

stck | Untitled-1.py | cre.py | list.py | uplod.py | dwn.py | del.py 2 | buc_del.py | u.py

ds > cre.py > ...

```python
import boto3
s3 = boto3.client('s3')
s3.put_bucket_replication(
    Bucket='my-boto-0',
    ReplicationConfiguration={
        "Role": "arn:aws:iam::123456789012:role/ReplicationRole",
        "Rules": [
            {
                "ID": "ReplicationRule1",
                "Priority": 0,
                "Status": "Enabled",
                "Filter": {"Prefix": ""},
                "Destination": {
                    "Bucket": "arn:aws:s3:::my-target-boto",
                    "StorageClass": "STANDARD"
                },
                # Required for current S3 replication
                "DeleteMarkerReplication": {
                    "Status": "Disabled"  # or "Enabled" if you want to replicate delete markers
                }
            }
        ]
    }
)
print("Cross-region replication configured successfully.")
```

meet.google.com is sharing your screen. Stop sharing Hide

Ln 10, Col 30 Spaces: 4 UTF-8 CRLF {} Python 3.13.2 Go Live

Type here to search 25°C Light rain ENG IN 12:02 AM 8/13/2025

Replication rule list - S3 bucket

https://ap-south-1.console.aws.amazon.com/s3/management/my-boto-0/replication?region=ap-south-1&bucketType=general

Just a moment... | DeepSeek - Into the... | Console Home | Co... | Console Home | ap-... | AWS Certified Cloud... | Your learning plan -... | Trailhead | The fun... | Free tests, notes, pa... | DLAI - Learning Plat...

aws Search [Alt+S] Asia Pacific (Mumbai) Account ID: 2982-4940-9320 Sumith

Amazon S3 > Buckets > my-boto-0 > Replication rules

Replication configuration settings

Create replication job | Edit

Configuration settings affect all replication rules in the bucket.

Source bucket
my-boto-0

Source Region
Asia Pacific (Mumbai) ap-south-1

IAM role
s3crr_role_for_my-boto-0

Replication rules (1)

View details | Edit rule | Delete | Actions | Create replication rule

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. Learn more

Replication rule name	Status	Destination bucket	Destination Region	Priority	Scope	Storage class	Replica owner	Replication Time Control	KMS-encrypted objects (SSE-KMS or DSSE-KMS)	Replica modification sync
ReplicationRule1	Enabled	s3://my-target-boto	Asia Pacific (Mumbai) ap-south-1	0	Prefix:	Same as source	Same as source	Enabled	Do not replicate	Disabled

meet.google.com is sharing your screen. Stop sharing Hide

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Type here to search 25°C Light rain ENG IN 12:02 AM 8/13/2025

7. Server-Side Encryption (put_bucket_encryption)

What it does:

Encrypts all objects automatically when stored.

Why it's used:

Enhances security and compliance.

Steps:

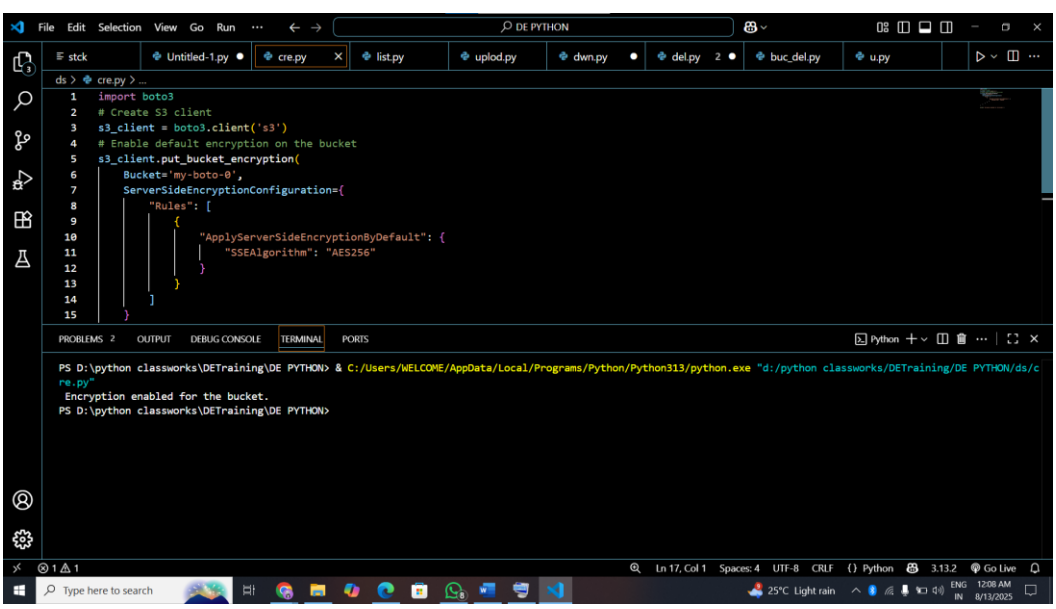
1. Use AES256 or AWS-KMS encryption.

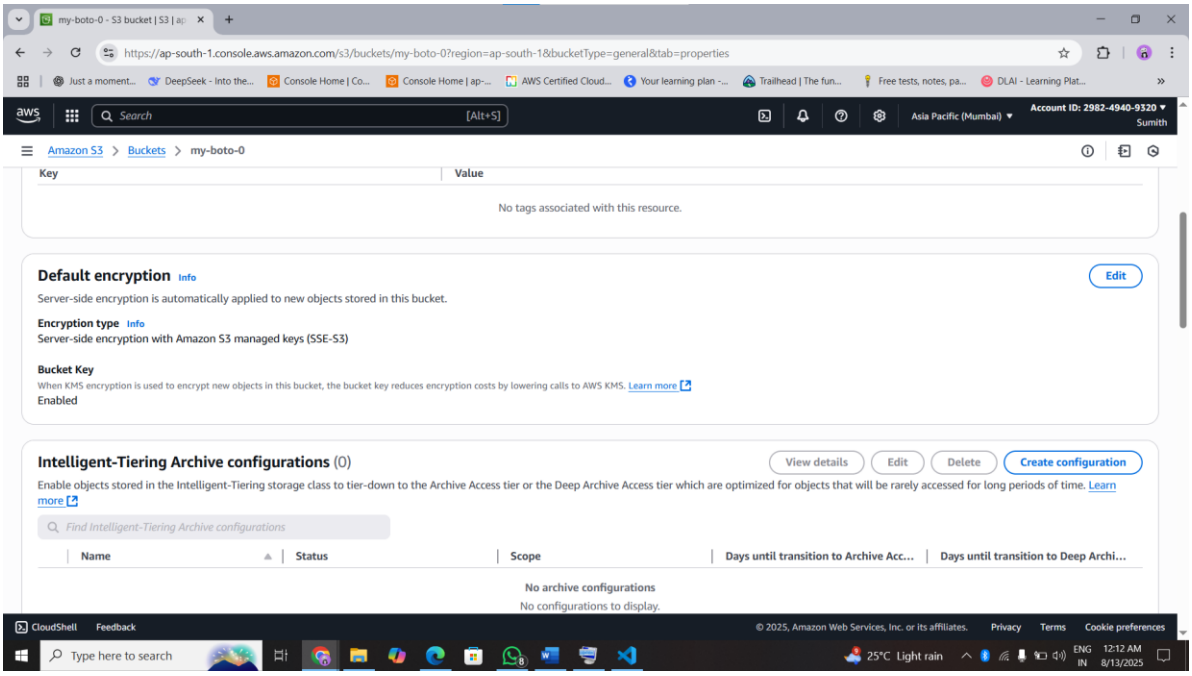
Code:

python

CopyEdit

```
s3_client.put_bucket_encryption(
    Bucket=BUCKET_NAME,
    ServerSideEncryptionConfiguration={
        "Rules": [
            {
                "ApplyServerSideEncryptionByDefault": {
                    "SSEAlgorithm": "AES256"
                }
            }
        ]
    }
)
print("✅ Encryption enabled for the bucket.")
```





---

8. Enable Logging (put_bucket_logging)

What it does:

Stores logs of requests to an S3 bucket.

Why it's used:

For auditing and monitoring.

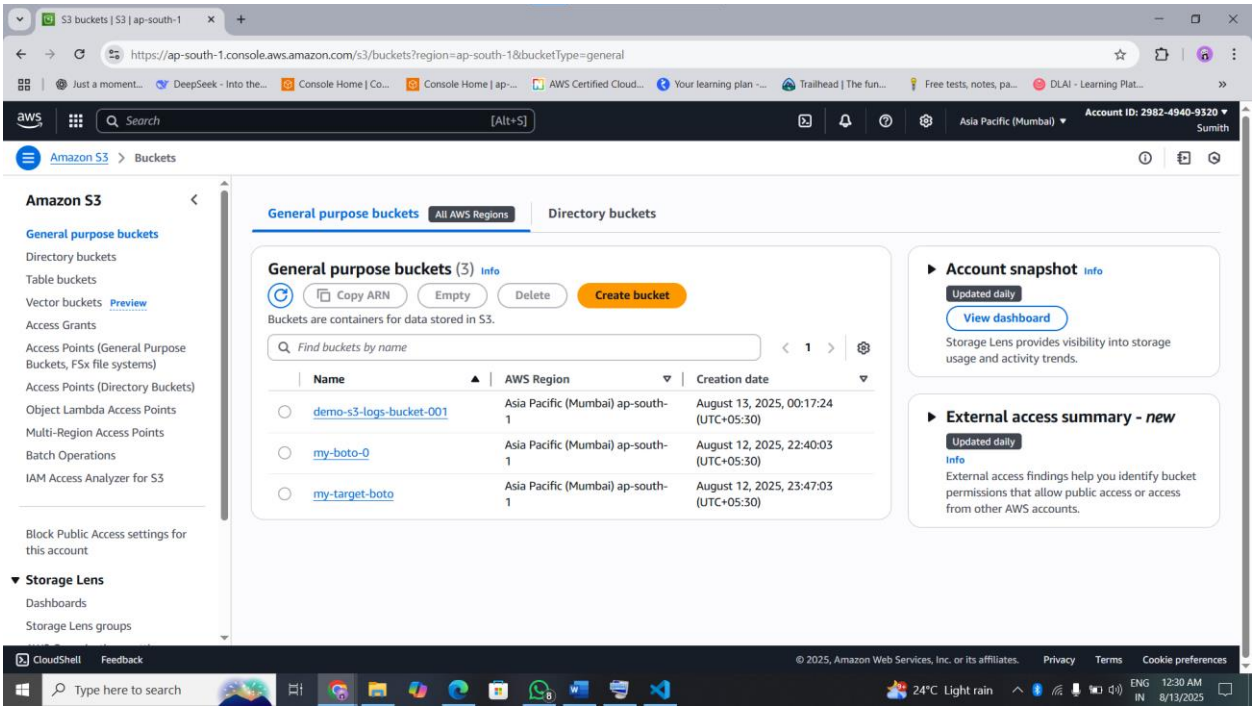
Steps:

1. Create a log bucket.
2. Enable logging to that bucket.

Code:

```
import boto3

# Create S3 client

s3_client = boto3.client('s3')

region = "ap-south-1"

SOURCE_BUCKET = "my-boto-0"

LOG_BUCKET = "demo-s3-logs-bucket-001"

# Create log bucket if not exists

try:

    s3_client.create_bucket(

        Bucket=LOG_BUCKET,

        CreateBucketConfiguration={"LocationConstraint": region}

    )
```

```
    print(f" Log bucket '{LOG_BUCKET}' created.")
except s3_client.exceptions.BucketAlreadyOwnedByYou:
    print(f"Log bucket '{LOG_BUCKET}' already exists.")
# Enable logging on source bucket
s3_client.put_bucket_logging(
    Bucket=SOURCE_BUCKET,
    BucketLoggingStatus={
        "LoggingEnabled": {
            "TargetBucket": LOG_BUCKET,
            "TargetPrefix": "logs/"
        }
    }
)
print(f"Logging enabled for '{SOURCE_BUCKET}' with logs going to '{LOG_BUCKET}'.")
```



```python
SOURCE_BUCKET = "my-boto-0"
LOG_BUCKET = "demo-s3-logs-bucket-001"

# Create log bucket if not exists
try:
    s3_client.create_bucket(
        Bucket=LOG_BUCKET,
        CreateBucketConfiguration={"LocationConstraint": region}
    )
    print(f" Log bucket '{LOG_BUCKET}' created.")
except s3_client.exceptions.BucketAlreadyOwnedByYou:
    print(f"Log bucket '{LOG_BUCKET}' already exists.")

# Enable logging on source bucket
```

```
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
Log bucket 'demo-s3-logs-bucket-001' already exists.
Logging enabled for 'my-boto-0' with logs going to 'demo-s3-logs-bucket-001'.
PS D:\python classworks\DETraining\DE PYTHON>
```

https://ap-south-1.console.aws.amazon.com/s3/buckets/my-boto-0?region=ap-south-1&bucketType=general&tab=properties

Amazon S3 > Buckets > my-boto-0

Server access logging — Edit

Log requests for access to your bucket. Use CloudWatch to check the health of your server access logging. Learn more

Server access logging	Log object key format
Enabled	logs/[YYYY]-[MM]-[DD]-[hh]-[mm]-[ss]-[UniqueString]

Destination bucket
s3://demo-s3-logs-bucket-001

AWS CloudTrail data events (0) Info — Configure in CloudTrail

Configure CloudTrail data events to log Amazon S3 object-level API operations in the CloudTrail console. Learn more

Name	Access
No data events	

9. Set CORS Rules (put_bucket_cors)

What it does:

Allows web apps from other domains to interact with your bucket.

Why it's used:

Needed for browser-based uploads or cross-origin requests.

Steps:

1. Define allowed methods, headers, origins.
2. Apply with put_bucket_cors.

Code:

python

CopyEdit

```
s3_client.put_bucket_cors(
    Bucket=BUCKET_NAME,
    CORSConfiguration={
        "CORSRules": [
            {
                "AllowedHeaders": ["*"],
                "AllowedMethods": ["GET", "POST", "PUT"],
                "AllowedOrigins": ["*"],
                "ExposeHeaders": ["x-amz-server-side-encryption"],
                "MaxAgeSeconds": 3000
            }
        ]
    }
)
print("✅ CORS rules applied to the bucket.")
```

```python
import boto3

# Create S3 client
s3_client = boto3.client('s3')

# Define bucket name
BUCKET_NAME = "my-boto-0"

# Apply CORS rules
s3_client.put_bucket_cors(
    Bucket=BUCKET_NAME,
    CORSConfiguration={
        "CORSRules": [
            {
                "AllowedHeaders": ["*"],
```

```
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/cre.py"
✅ CORS rules applied to the bucket.
PS D:\python classworks\DETraining\DE PYTHON>
```

https://ap-south-1.console.aws.amazon.com/s3/buckets/my-boto-0?region=ap-south-1&bucketType=general&tab=permissions

Amazon S3 > Buckets > my-boto-0

Cross-origin resource sharing (CORS)

Edit

The CORS configuration, written in JSON, defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. Learn more

Copy

```
[
  {
    "AllowedHeaders": [
      "*"
    ],
    "AllowedMethods": [
      "GET",
      "POST",
      "PUT"
    ],
    "AllowedOrigins": [
      "*"
    ],
    "ExposeHeaders": [
      "x-amz-server-side-encryption"
    ],
    "MaxAgeSeconds": 3000
  }
]
```

10. Configure Event Notifications (put_bucket_notification_configuration)

What it does:

Triggers events (SNS, SQS, Lambda) when objects are created/modified.

Why it's used:

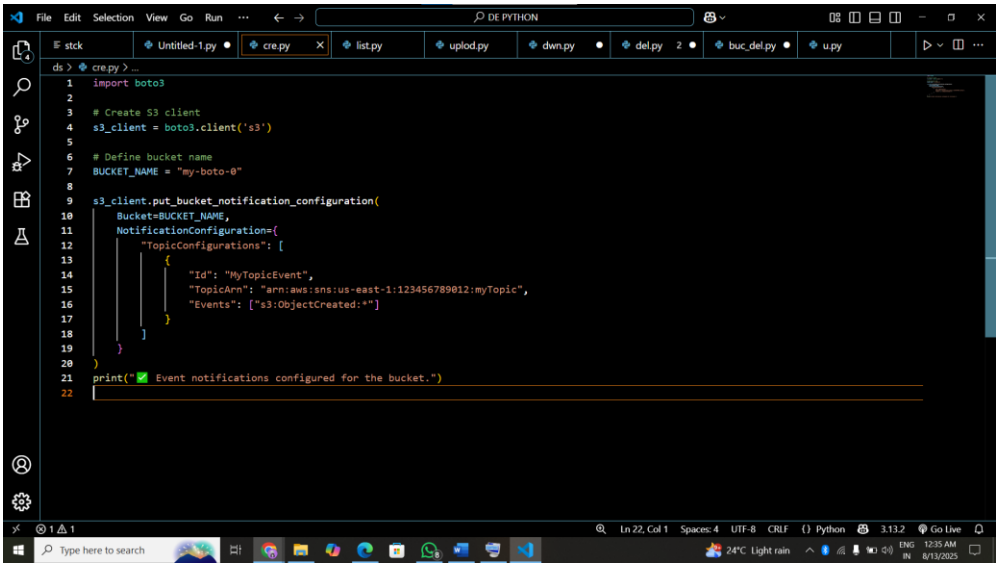
Automates workflows after uploads.

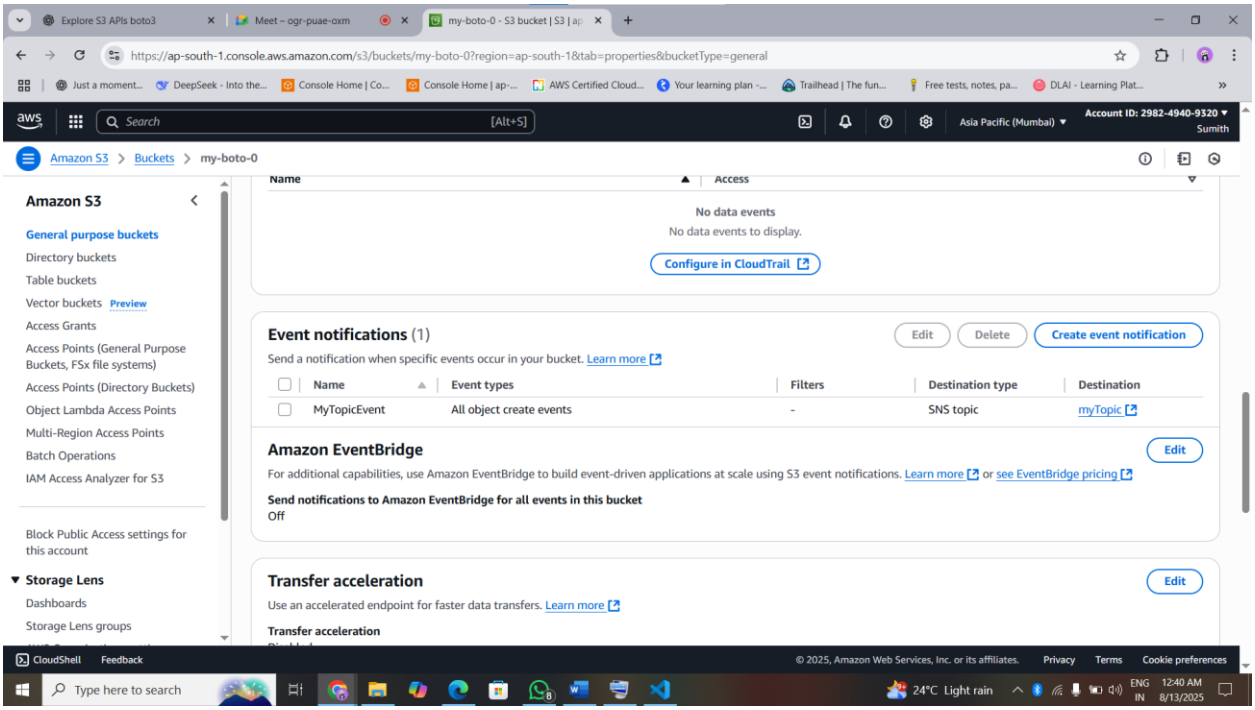
Steps:

1. Create SNS topic or Lambda.
2. Apply notification configuration.

Code:

```
s3_client.put_bucket_notification_configuration(
    Bucket=BUCKET_NAME,
    NotificationConfiguration={
        "TopicConfigurations": [
            {
                "Id": "MyTopicEvent",
                "TopicArn": "arn:aws:sns:us-east-1:123456789012:myTopic",
                "Events": ["s3:ObjectCreated:*"]
            }
        ]
    }
)
print("✅ Event notifications configured for the bucket.")
```





Task 1: Automate S3 Workflows Using AWS Lambda + boto3

Objective:

Set up an S3 → Lambda automation so that **whenever a file is uploaded to an S3 bucket, a Lambda function is triggered** to process it.

Steps

Step 1: Create the S3 Bucket

```
import boto3

REGION = "us-east-1"
BUCKET_NAME = "lambda-trigger-demo-bucket-001"

s3_client = boto3.client("s3", region_name=REGION)

# Create bucket
s3_client.create_bucket(
    Bucket=BUCKET_NAME,
    CreateBucketConfiguration={"LocationConstraint": REGION}
)
```

```
print(f"Bucket '{BUCKET_NAME}' created.")
```

Step 2: Create the Lambda Function

- This function will simply log the uploaded file's details.
- Save as **lambda_function.py**:

python

CopyEdit

```
import json

def lambda_handler(event, context):
    print("Event Received:", json.dumps(event))
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']
        print(f"File Uploaded: {key} in Bucket: {bucket}")
    return {
        'statusCode': 200,
        'body': json.dumps('File processed successfully')
    }
```

Step 3: Deploy Lambda using boto3

python

CopyEdit

```
import zipfile

LAMBDA_ROLE_ARN = "arn:aws:iam::123456789012:role/LambdaS3ExecutionRole"
LAMBDA_NAME = "S3FileUploadTrigger"

# Zip the lambda function
with zipfile.ZipFile("lambda_package.zip", "w") as zf:
    zf.write("lambda_function.py")

lambda_client = boto3.client("lambda", region_name=REGION)

with open("lambda_package.zip", "rb") as f:
```

```
    lambda_client.create_function(
        FunctionName=LAMBDA_NAME,
        Runtime="python3.9",
        Role=LAMBDA_ROLE_ARN,
        Handler="lambda_function.lambda_handler",
        Code={"ZipFile": f.read()},
        Timeout=30
    )

print(f" Lambda function '{LAMBDA_NAME}' created.")
```

Step 4: Configure S3 Event Notification to Trigger Lambda

python

CopyEdit

```
s3_client.put_bucket_notification_configuration(
    Bucket=BUCKET_NAME,
    NotificationConfiguration={
        "LambdaFunctionConfigurations": [
            {
                "LambdaFunctionArn": f"arn:aws:lambda:{REGION}:123456789012:function:{LAMBDA_NAME}",
                "Events": ["s3:ObjectCreated:*"]
            }
        ]
    }
)
print(" S3 Event Notification configured to trigger Lambda.")
```

Step 5: Test by Uploading a File

python

CopyEdit

```
s3_client.put_object(
    Bucket=BUCKET_NAME,
    Key="test_upload.txt",
    Body="This is a test file."
)
```

```
print(" Test file uploaded. Lambda should be triggered automatically.")
```

💡 **Outcome:** Every time you upload a file to the bucket, your Lambda will run and log file details.

Task 2: Compare S3 Storage Classes via boto3

Objective:

Upload the same file to S3 with different storage classes — **Standard**, **Intelligent-Tiering**, and **Glacier** — and compare them.

Steps

Step 1: Create a New Bucket

python

CopyEdit

```
BUCKET_NAME = "s3-storage-class-demo-001"

s3_client.create_bucket(
    Bucket=BUCKET_NAME,
    CreateBucketConfiguration={"LocationConstraint": REGION}
)
print(f" Bucket '{BUCKET_NAME}' created for storage class comparison.")
```

Step 2: Upload File with Different Storage Classes

python

CopyEdit

```
file_content = "Sample content for storage class comparison."

# Standard
s3_client.put_object(
    Bucket=BUCKET_NAME,
    Key="standard.txt",
    Body=file_content,
    StorageClass="STANDARD"
)
print(" Uploaded with Storage Class: STANDARD")

# Intelligent-Tiering
s3_client.put_object(
    Bucket=BUCKET_NAME,
    Key="intelligent-tiering.txt",
```

```
    Body=file_content,
    StorageClass="INTELLIGENT_TIERING"
)
print(" Uploaded with Storage Class: INTELLIGENT_TIERING")

# Glacier (deep archive storage)
s3_client.put_object(
    Bucket=BUCKET_NAME,
    Key="glacier.txt",
    Body=file_content,
    StorageClass="GLACIER"
)
print("Uploaded with Storage Class: GLACIER")
```

Step 3: Verify Storage Classes

python

CopyEdit

```
response = s3_client.list_objects_v2(Bucket=BUCKET_NAME)

for obj in response.get("Contents", []):
    head = s3_client.head_object(Bucket=BUCKET_NAME, Key=obj["Key"])
    print(f"File: {obj['Key']}, Storage Class: {head['StorageClass']}")
```

💡 **Outcome:**

- You'll have **three files stored with different cost/performance profiles**.
- Using head_object, you can confirm their storage classes.

Security & Compliance Audit Script

Objective:

1. **Scan for unencrypted objects** in S3 buckets.
2. **Identify publicly accessible buckets**.

We'll use boto3 to perform both checks.

Part 1: Scan for Unencrypted Objects

Logic:

- Loop through all buckets.
- For each object, check encryption status using head_object.
- If no ServerSideEncryption key exists in the response, mark it as unencrypted.

Code:

```
import boto3

REGION = "us-east-1"
s3_client = boto3.client("s3", region_name=REGION)

def scan_unencrypted_objects():
    print("\n Scanning for unencrypted objects...")
    buckets = s3_client.list_buckets()["Buckets"]

    for bucket in buckets:
        bucket_name = bucket["Name"]
        print(f"\nChecking bucket: {bucket_name}")

        # List objects in bucket
        try:
            objects = s3_client.list_objects_v2(Bucket=bucket_name)
        except s3_client.exceptions.ClientError as e:
            print(f"⚠ Could not list objects in {bucket_name}: {e}")
            continue

        if "Contents" not in objects:
            print("No objects found.")
```

```
        continue

    for obj in objects["Contents"]:
        key = obj["Key"]
        try:
            head = s3_client.head_object(Bucket=bucket_name, Key=key)
            if "ServerSideEncryption" not in head:
                print(f" Unencrypted: {key}")
            else:
                print(f" Encrypted: {key} ({head['ServerSideEncryption']})")
        except Exception as e:
            print(f"⚠ Error checking object {key}: {e}")

scan_unencrypted_objects()
```

Part 2: Identify Publicly Accessible Buckets

Logic:

- Loop through all buckets.
- Check get_bucket_policy_status to see if IsPublic is True.
- If public, flag it.

Code:

python

CopyEdit

```
def check_public_buckets():
    print("\nChecking for publicly accessible buckets...")
    buckets = s3_client.list_buckets()["Buckets"]

    for bucket in buckets:
        bucket_name = bucket["Name"]
        try:
            status = s3_client.get_bucket_policy_status(Bucket=bucket_name)
            if status["PolicyStatus"]["IsPublic"]:
                print(f" Public Bucket Found: {bucket_name}")
            else:
```

```
            print(f"Private Bucket: {bucket_name}")
        except s3_client.exceptions.ClientError as e:
            # Buckets without a policy will throw an error
            if "NoSuchBucketPolicy" in str(e):
                print(f" No policy found for {bucket_name} (default private).")
            else:
                print(f"⚠ Could not check bucket {bucket_name}: {e}")

check_public_buckets()
```

---

3 Combine Both Checks

You can merge them into a single **security_audit.py** script:

python

CopyEdit

```
if __name__ == "__main__":
    scan_unencrypted_objects()
    check_public_buckets()
```

Outcome:

- The script **flags unencrypted objects** so you can enable **SSE (Server-Side Encryption)**.
- It **identifies public buckets** so you can lock them down.

Part 1 – Transition Objects to Cheaper Tiers via Lifecycle Policies

We'll use put_bucket_lifecycle_configuration in **boto3**.

Steps

1. **Import boto3** and create an S3 client.
2. **Define a lifecycle policy** to transition objects to STANDARD_IA after 30 days.
3. **Apply the policy** to a specific bucket.
4. **Validate** the policy by checking get_bucket_lifecycle_configuration.

python

CopyEdit

```
import boto3
from botocore.exceptions import ClientError

# Step 1: Initialize S3 client
s3 = boto3.client('s3')

bucket_name = 'my-lifecycle-demo-bucket'

# Step 2: Define lifecycle configuration
lifecycle_config = {
    'Rules': [
        {
            'ID': 'MoveToIAAfter30Days',
            'Status': 'Enabled',
            'Filter': {'Prefix': ''},  # Applies to all objects
            'Transitions': [
                {
                    'Days': 30,
                    'StorageClass': 'STANDARD_IA'
                }
            ],
            'NoncurrentVersionTransitions': [
                {
                    'NoncurrentDays': 30,
                    'StorageClass': 'STANDARD_IA'
                }
```

```
        ]
      }
    ]
}

try:
    # Step 3: Apply lifecycle configuration
    s3.put_bucket_lifecycle_configuration(
        Bucket=bucket_name,
        LifecycleConfiguration=lifecycle_config
    )
    print(f"Lifecycle policy applied to bucket: {bucket_name}")

    # Step 4: Verify
    response = s3.get_bucket_lifecycle_configuration(Bucket=bucket_name)
    print("Current Lifecycle Configuration:", response)

except ClientError as e:
    print("Error:", e)
```

✅ **What it does:**

Any object in the bucket older than **30 days** automatically transitions to the **STANDARD_IA** storage class, reducing storage costs.

Part 2 – Generate Cost Reports using AWS Cost Explorer API

We'll use get_cost_and_usage in **boto3**.

Steps

1. **Create a Cost Explorer client**.
2. **Define time range & granularity** (e.g., daily, monthly).
3. **Specify metrics** like UnblendedCost or UsageQuantity.
4. **Fetch and print** the cost data.

python

CopyEdit

```
import boto3
from datetime import datetime, timedelta

# Step 1: Initialize Cost Explorer client
ce = boto3.client('ce')

# Step 2: Set time range (last 7 days)
end_date = datetime.utcnow().date()
start_date = end_date - timedelta(days=7)

# Step 3: Fetch cost report grouped by service
response = ce.get_cost_and_usage(
    TimePeriod={
        'Start': start_date.strftime('%Y-%m-%d'),
        'End': end_date.strftime('%Y-%m-%d')
    },
    Granularity='DAILY',
    Metrics=['UnblendedCost'],
    GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE'}]
)

# Step 4: Display results
print("AWS Cost Report:")
for result in response['ResultsByTime']:
    date = result['TimePeriod']['Start']
```

```
    for group in result['Groups']:
        service = group['Keys'][0]
        cost = group['Metrics']['UnblendedCost']['Amount']
        print(f"{date} - {service}: ${float(cost):.2f}")
```

✅ **What it does:**

This script retrieves **daily AWS costs** by service for the past **7 days**, so you can see how much S3 (or any other AWS service) costs you.

Step 1 – Create an S3 bucket & enable versioning

This ensures every file version is preserved when replaced or deleted.

python

CopyEdit

```
import boto3
from botocore.exceptions import ClientError

# AWS S3 client
s3 = boto3.client('s3')

bucket_name = "my-backup-bucket-unique-name"
region = "us-east-1"  # change to your region

# Create bucket
try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=bucket_name)
    else:
        s3.create_bucket(
            Bucket=bucket_name,
            CreateBucketConfiguration={'LocationConstraint': region}
        )
    print(f"Bucket '{bucket_name}' created successfully.")
except ClientError as e:
    print(f"Error creating bucket: {e}")

# Enable Versioning
try:
    s3.put_bucket_versioning(
        Bucket=bucket_name,
        VersioningConfiguration={'Status': 'Enabled'}
    )
    print(f"Versioning enabled on '{bucket_name}'.")
except ClientError as e:
    print(f"Error enabling versioning: {e}")
```

Step 2 – Automated uploads (Backup function)

This function uploads files to S3, automatically keeping old versions.

python

CopyEdit

```
import os

def backup_file(file_path, bucket):
    try:
        file_name = os.path.basename(file_path)
        s3.upload_file(file_path, bucket, file_name)
        print(f"Uploaded '{file_name}' to '{bucket}'.")
    except ClientError as e:
        print(f"Error uploading file: {e}")

# Example usage
backup_file("test_document.txt", bucket_name)
```

Step 3 – Lifecycle rule to archive old files to Glacier

Moves non-current versions to **S3 Glacier** after 30 days and deletes them after 365 days.

python

CopyEdit

```
lifecycle_config = {
    'Rules': [
        {
            'ID': 'ArchiveOldVersions',
            'Status': 'Enabled',
            'NoncurrentVersionTransitions': [
                {
                    'NoncurrentDays': 30,
                    'StorageClass': 'GLACIER'
                }
            ],
            'NoncurrentVersionExpiration': {
                'NoncurrentDays': 365
```

```
        }
      }
    ]
}

try:
    s3.put_bucket_lifecycle_configuration(
        Bucket=bucket_name,
        LifecycleConfiguration=lifecycle_config
    )
    print("Lifecycle policy applied successfully.")
except ClientError as e:
    print(f"Error applying lifecycle policy: {e}")
```

Step 4 – Cross-Region Replication for Disaster Recovery

This requires:

1. Versioning enabled **on both buckets**.
2. An **IAM role** with replication permissions.

Replication IAM Role Setup (One-time in AWS Console or via boto3)

- Create a role for S3 to assume.
- Attach AmazonS3FullAccess or least-privilege replication policy.
- Note the **Role ARN**.

Replication Setup Code:

python

CopyEdit

```
destination_bucket = "my-backup-bucket-dr"
destination_region = "us-west-2"

# Create destination bucket
if destination_region == "us-east-1":
    s3.create_bucket(Bucket=destination_bucket)
else:
    s3.create_bucket(
        Bucket=destination_bucket,
        CreateBucketConfiguration={'LocationConstraint': destination_region}
```

```
    )

# Enable versioning on destination bucket
s3.put_bucket_versioning(
    Bucket=destination_bucket,
    VersioningConfiguration={'Status': 'Enabled'}
)

# Replication configuration
replication_role_arn = "arn:aws:iam::123456789012:role/s3-replication-role"  # Replace

replication_config = {
    'Role': replication_role_arn,
    'Rules': [
        {
            'ID': 'ReplicationRule1',
            'Status': 'Enabled',
            'Priority': 1,
            'DeleteMarkerReplication': {'Status': 'Disabled'},
            'Filter': {'Prefix': ''},
            'Destination': {
                'Bucket': f"arn:aws:s3:::{destination_bucket}",
                'StorageClass': 'STANDARD'
            }
        }
    ]
}

try:
    s3.put_bucket_replication(
        Bucket=bucket_name,
        ReplicationConfiguration=replication_config
    )
    print("Cross-region replication configured.")
except ClientError as e:
```

```
    print(f"Error setting replication: {e}")
```

Step 5 – Test the Backup System

Upload a file and verify:

- File appears in the **primary bucket**.
- Lifecycle rule moves it to **Glacier** after 30 days.
- It is replicated to **destination bucket** in another region.

python

CopyEdit

```
backup_file("project_report.pdf", bucket_name)
```